Parallel Solutions for Sinkhorn Factorization

High Performance Computing for Data Science 2025/2026

Student: Francesco Magalini, mat: 247205

Contents

1. Introduction

Sinkhorn's theorem states the following: given any square matrix A of $n \times n$ **strictly positive** elements, there are two diagonal matrices D_1, D_2 with strictly positive diagonal elements such that $D_1 A D_2$ is **double stochastic**, that is each of its rows and columns sums to one.

$$\sum_i x_{ij} = \sum_j x_{ij} = 1$$

This process is also referred to as **Sinkhorn factorization** which has many practical applications, most notably in optimal transport, for example computing fast approximations to Wasserstein / Earth Mover's Distance. Machine learning is also another prominent usage of the factorization because it can be utilized for evaluating the difference between data distributions and permutations, which can improve the training phase of machine learning algorithms when maximum likelihood training may not be the preferred choice.

2. Computational Complexity

Analyzing the computational cost of the algorithm is important if we want to understand the parallelization procedure. The serial Sinkhorn-Knopp algorithm performs, at each iteration i, the following operations on an $n \times n$ matrix A:

- Row normalization: computing n row sums and dividing each element, $O(n^2)$ per iteration.
- Column normalization: computing n column sums and dividing each element, $O(n^2)$ per iteration.

Letting T denote the total number of iterations, the overall serial complexity is $O(T \cdot n^2)$. When matrix multiplication is used instead of the scaling formulation the cost rises to $O(T \cdot n^3)$, making parallelization even more necessary. In this report both variants are implemented and compared.

3. State of the Art

Researchers have conducted extensive works on this topic, in particular they have studied how this algorithm could be speed up through parallelization. As said before the applications that seems to be considered more useful are in the field of machine learning and optimal transport, in particular in the calculation of the Word Mover's Distance. The WMD measures semantic dissimilarity between two text documents, but classically calculating it is very expensive $O(V^3 \log(V))$ where V is the number of unique words in the document. Luckily the WMD can be approximated as an Earth Mover's Distance (EMD) for which the algorithmic complexity can be reduced to $O(V^2)$ and it can be solved using the Sinkhorn–Knopp algorithm. For example a paper by Tithi and Petrini brings first a theoretical contribution, by transforming the *"dense compute-heavy kernel to a sparse compute kernel and obtain 67× speedup"*. They also have shown an implementation of their algorithm using OpenMP and C++ which is 700× faster than the naive Python implementation. This program was run with 96 cores on a state of the art Intel® 4-sockets Cascade Lake machine. Another paper by the same authors, a year later, provided a new version that brings additional theoretical speedups by leveraging the architecture of both conventional CPUs and emerging sparse-friendly hardware (the Intel Programmable Integrated Unified Memory Architecture aka PIUMA).

A different paper by Sun, Luo, Jiang, Zhang and Li implements an optimized version of the Sinkhorn-Knopp algorithm, named COFFEE, specifically tailored for HPC environments. They implemented various optimizations including: a redesigned column rescaling approach to improve memory locality and reduce cache misses, data blocking for cache-friendly matrix partitioning, custom SIMD vectorized micro kernels for high instruction level parallelism, and advanced MPI enhancements such as balanced intranode Reduce and pipelined/overlapped hierarchical Ring Allreduce.They were able to achieve substantial performance improvements. Experiments on the Tianhe-1 supercomputer (up to 64 nodes/576 cores) show up to 7.5× speedup on a single node and up to 2.9× in multi-node settings compared to standard MPI based implementations.

4. Serial Algorithm

Stated the problem, there is a simple but effective algorithm called the Sinkhorn–Knopp algorithm (or just Sinkhorn algorithm) which takes an iterative approach, meaning that the solutions are initially approximative and subsequently improved after each iteration, and the i-th solution is derived from the previous one.

Below is presented the pseudocode of the algorithm

```
function getError(vector)
  return max(abs(vector .- 1))
end

function sinkhorn(A, tolerance = 1e-6, maxIterations = 100)
 # Sinkhorn factorization A = D1 S D2 where D1 and D2 are diagonal and S doubly
 # stochastic using Sinkhorn-Knopp algorithm which consists in iterative rows
 # and columns sums normalizations.
 n = size(A)
 D1, D2 = vector(1,n), vector(1,n) # initialize as vectors of length n and value 1
 S = A
 iter, err = 0, +Inf
 while ((err > tolerance) && (iter < maxIterations))
   rowSums = sum(S, dims = 2) # return a vector of row sums
   D1 = D1 .* rowSums
   S = Diagonal(1 ./ rowSums) * S # Diagonal returns the diagonal matrix from a
```

```
vector
   columnSums = sum(S, dims = 1) # return a vector of column sums
   D2 = columnSums .* D2
   S = S * Diagonal(1 ./ columnSums)
   iter += 1
   err = max(getError(rowSums), getError(columnSums))
 end
 return S, Diagonal(D1), Diagonal(D2), iter
end
```

Successively we implemented this idea in C++ as shown below and measured the execution time. As expected this naive implementation is fairly slow given the fact that not only it is not parallelized but it is also based on matrix multiplication.

```
// operations executed during each iteration
vector<double> rowSum = sumAlong(matrix, 1);
D1 = vectorMultiply(D1, rowSum);
matrix = matrixMultiply(diagonal(reciprocal(rowSum)), matrix);
vector<double> columnSum = sumAlong(matrix, 0);
D2 = vectorMultiply(columnSum, D2);
matrix = matrixMultiply(matrix, diagonal(reciprocal(columnSum)));
double row_error = getError(matrix, 1);
double column_error = getError(matrix, 0);
error = max(row_error, column_error);
iter++;
```

Note: Executed on a HPC Cluster with a Linux CentOS 7 Operative System and PBS Professional.

```
#!/bin/bash
#PBS -l select=1:ncpus=3:mem=10gb
# set max execution time
#PBS -l walltime=01:30:00
# imposta la coda di esecuzione
#PBS -q shortCPUQ
# ARG1 = matrix size
./HPC/code/serial/executables/linux/serial "$ARG1"
```

Matrix Size (*n*)	**Execution Time (seconds)**
1000	~124s
1200	~280s
1400	~407s (= ~7min)
1600	~729s (= 12min)
2000	~1756s (= 29min)
2400	~2140s (= 35min)
3600	~4717s (= ~79min)

5. Parallel Algorithm

Given the single process approach we can see how it is particularly suitable for parallelization as operations involving rows and columns can be done independently. As a first approach however we simply parallelized the matrix multiplication using OpenMP directives.

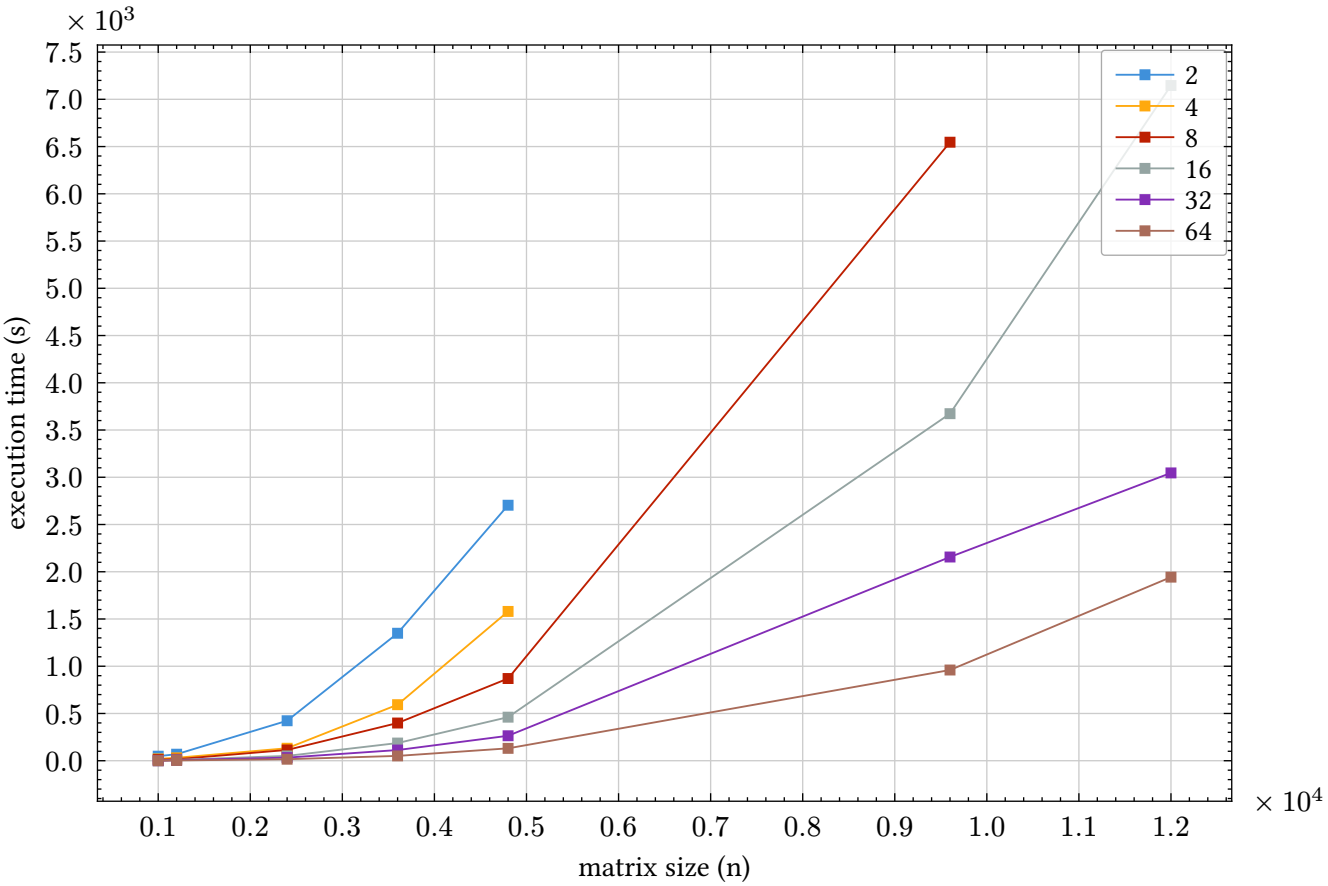
```
DMatrix matrixMultiplyOpenmp(DMatrix A, DMatrix B)
    // Multiplies two matrices A and B
    {
        int n = A.size();
        DMatrix C(n, vector<double>(n, 0.0));
        #pragma omp parallel for schedule(static)
        for (int i = 0; i < n; i++)
        {
            for (int k = 0; k < n; k++)
            {
                double temp = A[i][k];
                for (int j = 0; j < n; j++)
                {
                    C[i][j] += temp * B[k][j];
                }
            }
        }
        return C;
    }
```

We can also notice how the loops and the formulas were slightly rearranged in order to leverage a cache friendly access pattern, thus providing a further practical speedup. This is because in the code `C[i][j] += temp * B[k][j]` the matrix `B` is accessed row-wise. Since a matrix is also stored in memory row-wise, this means it is accessed sequentially and it translates to a very good cache locality.

Below we can see the execution times of this solution. We can immediately notice how it is faster than the sequential solution, but other than that we can observe the speedup offered by adding more threads. Of course, there is also in certain cases a possible diminishing return effect due to the overhead brought by multi threading, but in general by doubling the number of threads the execution time halves.

```
#!/bin/bash
# change the number of cpus as needed, more threads should require more cpus
#PBS -l select=1:ncpus=64:mem=240gb
# set max execution time
#PBS -l walltime=03:00:00
# imposta la coda di esecuzione
#PBS -q shortCPUQ
# ARG1 = matrix size
# ARG2 = n threads (OpenMP)
# ARG3 = algorithm type (matmul based or not)
./HPC/code/parallel/executables/linux/parallel "$ARG1" "$ARG2" "$ARG3"
```

Size (n)	2 threads	4 threads	8 threads	16 threads	32 threads	64 threads
1000	~48s	~19.5s	~12s			
1200	69s	31s	~16s	~9s	6s	3s
2400	423s	132s	~114s	51s	33s	16s
3600	1349s	593s	399s	187s	113s	51s
4800	2704s	1580s	870s	461s	264s	131s
9600			6546s	3673s	2156s	960s
12000				7146s	3046s	1943s



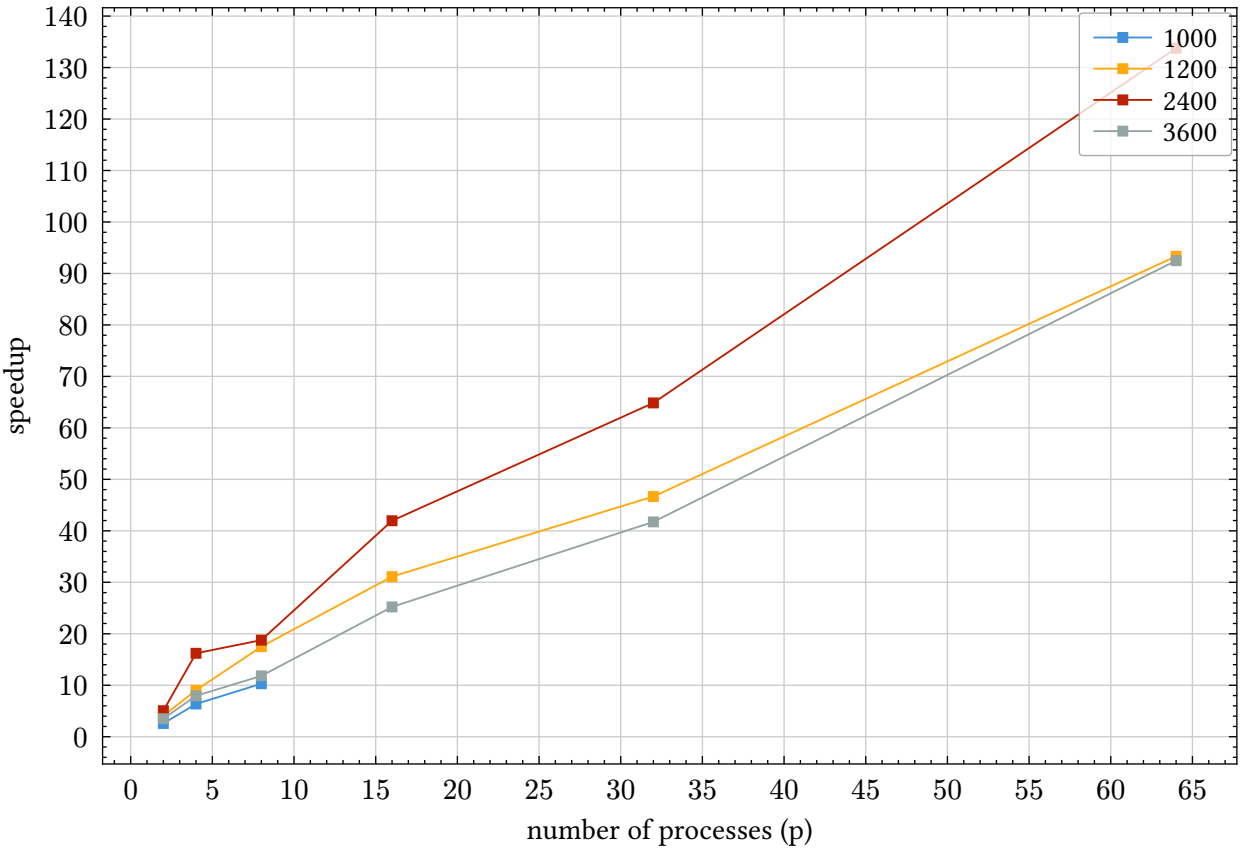
By analyzing the speedup with formula $S = T_{\text{serial}}/T_{\text{parallel}}$ end efficiency with formula $E = S/p$ we can notice interesting insights. First of all efficiently is $\gg 1$, this is due to the fact that the baseline version is a naive and unoptimized implementation with cubic computational complexity $O(n^3)$, while the parallel version brings not only multi threading but also cache optimization strategies. This can also be noticed in the last line of the speedup measurements table where, despite an increasing trend both in the horizontal and vertical direction, this last line stands as an outlier most likely because that specific matrix size is sub optimal for the cache page size. It can also be said that besides some exceptions generally the efficiency slightly decreases when the number of threads increases, but overall this solution presents very favorable scalability properties as it is based on matrix multiplication.

Speedup

Size (n)	2 threads	4 threads	8 threads	16 threads	32 threads	64 threads
1000	2.58	6.36	10.3			
1200	4.06	9.03	17.5	31.11	46.67	93.33
2400	5.06	16.21	18.77	41.96	64.85	133.75
3600	3.5	7.95	11.82	25.22	41.74	92.49

Efficiency

Size (n)	2 threads	4 threads	8 threads	16 threads	32 threads	64 threads
1000	1.29	1.59	1.28			
1200	2.03	2.26	2.19	1.94	1.46	1.46
2400	2.53	4.05	2.35	2.62	2.03	2.09
3600	1.75	1.99	1.48	1.58	1.30	1.44



We implemented another version of the algorithm based on a simple scaling operation instead of matrix multiplication. This algorithm is able to tackle very high matrix sizes (*several times bigger than the previous version*) in a considerable amount of time. The implementation is optimized using OpenMP as well and we tried to keep in mind cache locality also in this case. Despite this, this

version of the algorithm doesn't presents the strong scalability properties of the previous version. The most likely explanation is that the bottleneck here is not computation. Because the complexity here is $O(n^2)$ and not $O(n^3)$, for every input value there is a single scaling operation. Thus the real pain point becomes loading values from memory. Adding more threads despite giving small benefits, doesn't help like in the matrix multiplication case. Even if each thread operates on a subset of the matrix the memory has to stream even more values per second to each thread and so contention problems could arise and slow down the whole operation, a phenomenon that was not relevant in the previous case. A better data structure than `vector<vector<double>>` could also improve the situation but the underlying problem still remains.

Both relevant pieces of code and execution times are presented below.

```
DMatrix matrixNormalizeOpenmp(DMatrix matrix, vector<double> v, int axis)
    {
        int n = matrix.size();
        DMatrix C(n, vector<double>(n, 0.0));
        if (axis == 0)
        { // Normalize along columns
            #pragma omp parallel for
            for (int i = 0; i < n; i++)
            {
                for (int j = 0; j < n; j++)
                {
                    C[i][j] = matrix[i][j] / v[j];
                }
            }
        }
        else if (axis == 1)
        { // Normalize along rows
            #pragma omp parallel for
            for (int i = 0; i < n; i++)
            {
                double v_i = v[i];
                for (int j = 0; j < n; j++)
                {
                    C[i][j] = matrix[i][j] / v_i;
                }
            }
        }
        return C;
    }
```

Matrix Size (*n*)	**1 thread**	**2 threads**	**4 threads**	**8 threads**	**16 threads**	**32 threads**	**64 threads**
16000	70s	52s	38s	33s	34s	21s	27s
32000	329s	256s	242s	208s	194s	183s	135s
64000	1698s	1691s	1589s	769s	1039s	897s	668s

Speedup

Matrix Size (n)	2 threads	4 threads	8 threads	16 threads	32 threads	64 threads
16000	1.35	1.84	2.12	2.05	3.33	2.59
32000	1.29	1.36	1.58	1.70	1.80	2.44
64000	1	1.07	2.21	1.63	1.89	2.54

Efficiency

Matrix Size (n)	2 threads	4 threads	8 threads	16 threads	32 threads	64 threads
16000	0.68	0.46	0.27	0.13	0.10	0.04
32000	0.65	0.34	0.20	0.11	0.06	0.04
64000	0.5	0.27	0.28	0.10	0.06	0.04

5.1. Data Dependencies

Attesting the presence of unresolved data dependencies in OpenMP is highly important for ensuring a smooth and correct execution. When a statement result depends on the other, that is the execution order matters, it is possible to get wrong results or a non deterministic behavior if these aspects are not addressed properly. First let's take a look at the `matrixMultiplyOpenmp` function. The parallelism involves the outermost for loop over `i`, where each threads runs independently the full inner loops over `k` and `j` for their respective assigned rows. Also it is clear that there are no race conditions since each thread writes only to rows of `C` it owns (`C[i][...]` for its assigned range of `i`) so it is not possible for two threads ever write to the same `C[i][j]`. Now let's formalize what it has just been discussed, we focus on the two main statements

- **S1:** `temp = A[i][k]`
- **S2:** `C[i][j] += temp * B[k][j]`

Memory Location	Earlier Statement	Later Statement	Loop Carried	Dataflow
`temp`	S1, `(i,k)`, write	S2, `(i,k,j)`, read	No	Flow
`C[i][j]`	S2, `(i,k,j)`, read	S2, `(i,k+1,j)`, write	Yes, but carried by the `k` loop only	Flow
`A[i][k]`	S1, `(i,k)`, read	-	No other access	None
`B[k][j]`	S2, `(i,k,j)`, read	-	No other access	None

Now regarding the `matrixNormalizeOpenmp` function the situation is not only similar but even simpler. Again each threads operates on its preassigned sets of rows in a parallelized for loop over `i`. The inner loop over `j` runs independently for each thread, writings results into `C[i][j]` by reading `matrix[i][j]` and `v[j]` (or `v[i]`). As the previous case the benefit of parallelization are straightforward because the threads operate independently and there is no data race, `C[i][j]` is written by exactly one thread (whichever owns row `i`) and `matrix` and `v` are only read and never modified. Now for the more formal proof:

- **axis == 0: S1:** `C[i][j] = matrix[i][j] / v[j]`
- **axis == 1: S1′:** `v_i = v[i]` , **S2′:** `C[i][j] = matrix[i][j] / v_i`

Memory Location	Earlier Statement	Later Statement	Loop Carried	Dataflow
`C[i][j]`	S1, `(i,j)`, write	no later read of `C` in this function	No	None
`matrix[i][j]`	S1, `(i,j)`, read	-	No	None
`v[j]`	S1, `(i,j)`, read	-	No	None

Memory Location	Earlier Statement	Later Statement	Loop Carried	Dataflow
`v_i`	S1′, `i`, write	S2′, `(i,j)`, read	No, same `i` and doesn't change across `j`	Flow
`C[i][j]`	S2′, `(i,j)`, write	-	No	None
`matrix[i][j]`	S2′, `(i,j)`, read	-	No	None
`v[i]`	S1′, `i`, read	-	No	None

6. Distributed Parallel Algorithm

In a multi node context the workload needs to be distributed beforehand and there needs to be a minimal amount of communication between nodes. We decided to adopt the well known Forster's methodology also known as PCAM (Partitioning, Communication, Agglomeration, Mapping). PCAM is a simple design recipe that breaks down the process into four phases to ensure efficient parallelism, and it could generally be applied to almost any algorithm that can be parallelized.

6.1. Distributed Sinkhorn-Knopp

1. After generating a $n \times n$ matrix the algorithm proceeds by equally partitioning all the n rows among the p processes (in our case n must be divisible by p) using the function *MPI_Scatter*. So each process operates on a submatrix that has $\frac{n}{p}$ rows and n columns. Additionally the overall matrix is "flattened" and turned into a vector during this process for communication simplicity.
2. The algorithm executes the following loop for a pre-determined number of iterations
3. First each process computes independently the sums along its submatrix rows, and divides each element of the row by this sum value.
4. Then each process computes the sums along its submatrix columns. Then these vectors are summed together among all processes to obtain the global sums of the columns. This is done using *MPI_Allreduce* with the *MPI_SUM* flag. After obtaining the vector of global column sums each processes divides independently its columns by the corresponding value. After this we go back to the beginning of the loop.
5. After exiting the loop, using *MPI_Gather* all the sub matrices of each process are combined into one, as well as the *D1* and *D2* vectors that were calculated in the loop (*see the introduction section for a reference of the meaning of these variables*).

In the table below are noted the execution times of the program described above. First we can notice how it is a bit slower compared to the purely OpenMP version. This is because the MPI processes are

scattered among different nodes (*even when they are packed the communication is faster but still in place*) so naturally communication creates an overhead, but it must be said that a multi node architecture allows to tackle inputs that may not fit in the memory or be able to be processed by a single node. We can also notice how by adding more processes the computation becomes faster, for example the execution times with 4 processes are half compared to those with only 2. However with 8 processes there is no longer a speedup. We suspect that the slowdown caused by communication becomes more relevant than the theoretical speedup brought by adding more processes.

Note: the execution was carried out using `-l place=pack` and 4 OpenMP threads

```
#!/bin/bash
# change the number of cpus as needed, more threads should require more cpus
#PBS -l select=16:ncpus=4:mem=80gb -l place=pack
# set max execution time
#PBS -l walltime=03:00:00
# imposta la coda di esecuzione
#PBS -q shortCPUQ
module load OpenMPI/4.1.6-GCC-13.2.0
echo "ARG2: $ARG2"
mpirun -n 16 ./HPC/code/distributed/executables/distributed "$ARG1" "$ARG2" "$ARG3"
```

Matrix Size (*n*)	**2 MPI processes**	**4 MPI processes**	**8 MPI processes**	**16 MPI processes**
16000	74s	18s	15s	16s
32000	219s	106s	119s	103s
64000	2646s	614s	510s	622s

Speedup

Matrix Size (*n*)	**2 MPI processes**	**4 MPI processes**	**8 MPI processes**	**16 MPI processes**
16000	0.95	3.89	4.67	4.37
32000	1.50	3.10	2.76	3.19
64000	0.64	2.77	3.33	2.73

Efficiency *(considering as p the MPI processes alone)*

Matrix Size (*n*)	**2 MPI processes**	**4 MPI processes**	**8 MPI processes**	**16 MPI processes**
16000	0.48	0.97	0.58	0.21
32000	0.75	0.78	0.35	0.20
64000	0.32	0.69	0.42	0.17

Note: the execution was carried out using `-l place=scatter` and 4 OpenMP threads

```
#!/bin/bash
# change the number of cpus as needed, more threads should require more cpus
#PBS -l select=16:ncpus=4:mem=120gb -l place=scatter
# set max execution time
#PBS -l walltime=03:00:00
# imposta la coda di esecuzione
#PBS -q shortCPUQ
module load OpenMPI/4.1.6-GCC-13.2.0
echo "ARG2: $ARG2"
mpirun -n 16 ./HPC/code/distributed/executables/distributed "$ARG1" "$ARG2" "$ARG3"
```

Matrix Size (*n*)	**2 MPI processes**	**4 MPI processes**	**8 MPI processes**	**16 MPI processes**
16000	52s	25s	22s	18s
32000	213s	148s	115s	95s
64000	1313s	751s	740s	437s

7. Possible Improvements

We believe it is possible to further improve these solutions while maintaining an overall conceptual simplicity. The first factor are CPU intensive operations which can be further sped up by leveraging both SIMD instruction sets such as AVX2 or AVX-512 which are supported by most of the modern processors as well as a better exploiting of cache-locality. The second factor is a better orchestration between MPI processes to reduce node communication to a minimum, for example it should be explored the idea of compressing the vectors so the data sent around and thus the idle time as well as the pressure on the memory is decreased. Another beneficial measure that should be introduced is a way to measure execution times of pieces of code like with local debuggers but by making it work in a PBS environment. This way it is possible to pinpoint with more accuracy which aspects represent a bottleneck during execution and handle them with more awareness. Lastly a complete overhaul of the $O(n^2)$ algorithm could be considering by taking into account the memory bottleneck that penalizes scalability, but doing so requires a careful planning and analysis of the hardware architectures and state of the art algorithms.

8. Conclusion

During this project we have learnt how to implement a parallel algorithm in cpp using both OpenMP directives and MPI functions by implementing working solutions for the Sinkhorn-Knopp algorithm. In particular, we first explored how to transform a computationally intensive iterative matrix scaling method into efficient parallel version using OpenMP. We implemented a shared memory approach that leverages compiler directives to parallelize loops and distribute workload among threads while carefully managing synchronization and avoiding race conditions, all while keeping an eye on cache efficiency. In contrast, with MPI, we developed a distributed solution in which data is partitioned across multiple processes, requiring explicit communication through message passing to ensure correctness at each iteration. Throughout this process, we gained practical experience in workload decomposition, process communication and performance optimization. Overall, this project strengthened our knowledge of parallel computing concepts and

provided hands on experience in applying them to a real numerical algorithm, in particular by using industry standard technologies such as PBS, MPI and OpenMP.

9. References

1. Sinkhorn Theorem
2. Earth Mover Distance
3. Tithi, Petrini: “An Efficient Shared-memory Parallel Sinkhorn-Knopp Algorithm to Compute the Word Mover’s Distance”
4. Tithi, Petrini: “A New Parallel Algorithm for Sinkhorn Word-Movers Distance Using Fused SDDMM-SpMM on PIUMA and Xeon CPU”
5. Sun, Luo, Jiang, Zhang and Li: “COFFEE: Cross-Layer Optimization for Fast and Efficient Executions of Sinkhorn-Knopp Algorithm on HPC Systems”